

Lecture 15

Monday, April 6, 2026 7:23 PM

Today: Gradient Descent (and its variants)
for Linear Regression

Loss function: $L(\theta)$
 $\downarrow$ parameter
 function

$$\theta^* = \operatorname{argmin} L(\theta)$$

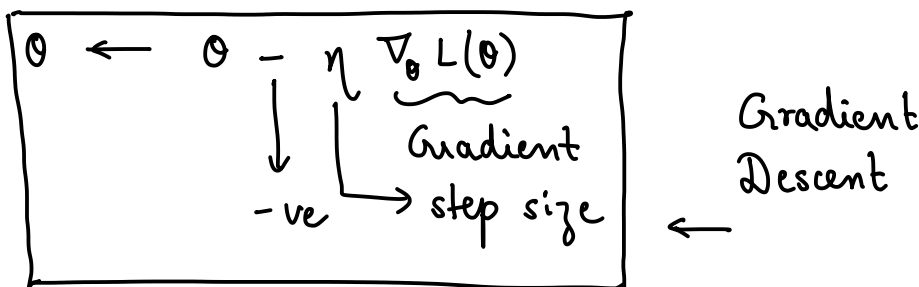
Gradient : $L: \mathbb{R}^d \rightarrow \mathbb{R}$

$$\nabla_{\theta} L = \begin{pmatrix} \frac{\partial L}{\partial \theta_0} \\ \frac{\partial L}{\partial \theta_1} \\ \vdots \\ \frac{\partial L}{\partial \theta_d} \end{pmatrix} \in \mathbb{R}^d$$

$$\left. \begin{array}{l} L(\theta) = \theta_1^2 + 2\theta_2 \\ \nabla_{\theta} L = \begin{pmatrix} 2\theta_1 \\ 2 \end{pmatrix} \\ \theta_1 = 3 \quad \theta_2 = 5 \\ (3, 5) \quad (3+8, 5) \\ \quad \quad (3-8, 5) \end{array} \right\}$$

$\frac{\partial L}{\partial \theta_i}$ measure of how L increases if we nudge the i^{th} dimension keeping every other dimension constant

- Gradient points in the direction of the steepest ascent
- Moves in the direction of $-\nabla_{\theta} L(\theta)$



Linear Regression

$d-1$ features $x = [1 \text{ --- }] \leftarrow 1^{\text{st}} \text{ sample}$

$$\begin{bmatrix} 1 \\ \vdots \\ i \\ \vdots \\ n \end{bmatrix} \left\{ \begin{array}{l} \leftarrow i^{\text{th}} \text{ sample} \\ \leftarrow n^{\text{th}} \text{ sample} \end{array} \right. \quad \begin{array}{l} n \times d \text{ matrix} \end{array}$$

$y^{(i)}$ — value corresponding to sample i

$$L(\theta) = \frac{1}{2n} \|x\theta - y\|^2 = \frac{1}{2n} \|y - x\theta\|^2 \leftarrow$$

constant, added for convenience

$$\theta^* = \underset{\theta}{\operatorname{argmin}} L(\theta)$$

— θ^* can be found using linear algebra

$$\nabla_{\theta} L(\theta) = \frac{1}{n} \underbrace{x^T}_{1 \times 1} \underbrace{(x\theta - y)}_{\underbrace{n \times d \times d \times 1}_{n \times 1}} = \frac{1}{n} \underbrace{x^T}_{1 \times 1} \underbrace{(\hat{y} - y)}_{n \times 1}$$

$\underbrace{(1 \times 1)(d \times n)(n \times 1)}_{d \times 1}$

Proof omitted (verify) $\theta \leftarrow \theta - \eta \nabla_{\theta} L(\theta)$

$$\theta \leftarrow \theta - \frac{\eta}{n} x^T (x\theta - y)$$

(Batch) Gradient Descent for linear regression

- $x\theta$: $O(nd)$ multiplications
 - $x\theta - y$: $O(n)$ subtractions
 - $x^T(x\theta - y)$: $O(nd)$ multiplication
- $O(nd)$ computations needed for each step

Flowing market : $n = 10^7$ $d = 50$: 5×10^8
 Image processing : $n = 10^6$ $d = 10^5$: 10^{11}

Memory requirement is also (prohibitively) large.

Stochastic Gradient Descent

$$\Rightarrow L(\theta) = \frac{1}{2n} \|x\theta - y\|^2$$

Core Idea

$$\underbrace{L^{(i)}(\theta)} = \frac{1}{2} (\hat{y}^{(i)} - y^{(i)})^2$$

contribution of the i^{th} sample to the loss

$$\Rightarrow \nabla_{\theta} L^{(i)}(\theta) = (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$$

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L^{(i)}(\theta) = \theta - \eta (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$$

- ① how expensive is this step — $O(d)$
- ② why does this make sense?

$$\begin{aligned} \text{① } \hat{y}^{(i)} &: O(d) \leftarrow \\ \hat{y}^{(i)} - y^{(i)} &: O(1) \\ (\hat{y}^{(i)} - y^{(i)}) x^{(i)} &: O(d) \leftarrow \end{aligned}$$

Overall: $O(d)$

$\Rightarrow i$ — uniformly at random

$$\text{② } \mathbb{E} [\underbrace{\nabla_{\theta} L^{(i)}(\theta)}] = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} L^{(i)}(\theta) = \nabla_{\theta} L(\theta)$$

(proof omitted)

On average, stochastic gradient points in the right direction

SGD Gradient is unbiased

$$\theta \leftarrow \theta - \underbrace{\eta_t}_{\text{learning rate}} \nabla_{\theta} L^{(i)}(\theta)$$

decrease with time : $\eta_t = \frac{1}{\sqrt{t}}$

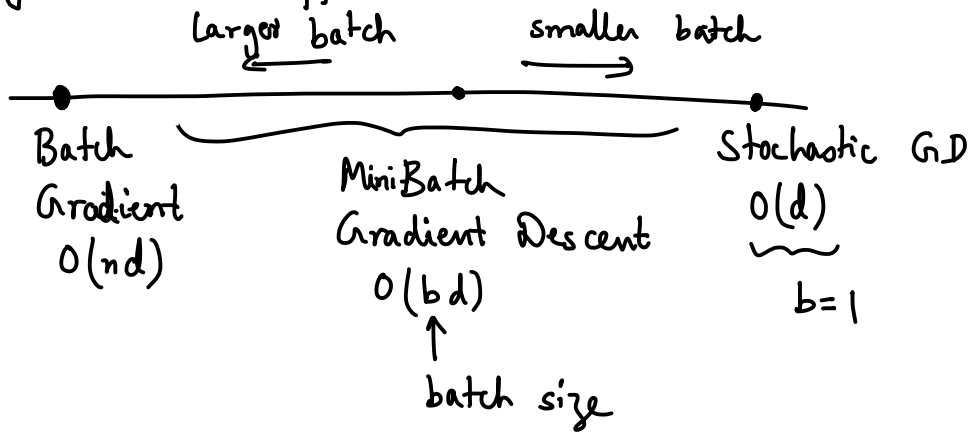
SGD algorithm

1. Initialize θ
2. For each epoch (pass through all n datapoints)
 - a) shuffle the data (permutation of $1:n$)
 - b) for each sample $(x^{(i)}, y^{(i)})$ in the shuffled order

Compute $\hat{y}^{(i)} = x^{(i)T} \theta$

Update $\theta \leftarrow \theta - \eta (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$
3. Repeat until

Why do we shuffle?



Mini-batch Gradient Descent

Pick B data-points (uniformly at random)

$$\nabla_{\theta} L_B(\theta) = \frac{1}{B} \sum_{i \in B} (\hat{y}^{(i)} - y^{(i)}) x^{(i)} = \frac{1}{B} X_B^T (X_B \theta - y_B)$$

X_B : X restricted to row corresponding to the mini-batch

$$\theta \leftarrow \theta - \frac{\eta}{B} X_B^T (X_B \theta - y_B)$$

Complexity : $O(Bd)$

$$L(\theta) = \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 \leftarrow \text{RSS} \quad \nabla_{\theta} L(\theta) = \frac{1}{n} X^T (\hat{y} - y)$$

$$L^{(i)}(\theta) = (\hat{y}^{(i)} - y^{(i)})^2 \quad \nabla_{\theta} L^{(i)}(\theta) = (\hat{y}^{(i)} - y^{(i)}) x^{(i)}$$

Formal implementation of Mini-batch

- ~~Initizi~~ Initialize θ
- For each epoch
 - a) shuffle data
 - b) partition data into mini-batches
 $B_1, B_2, \dots, B_{\lceil \frac{n}{B} \rceil}$, each of size B

c) For each mini-batch

$$\theta \leftarrow \theta - \frac{\eta}{B} X_{B_k}^T (X_{B_k} \theta - y_{B_k})$$

- Repeat until ...